

## Problem Statement

The fundamental challenge posed by the Prisoner's Dilemma is the structural conflict between **individual rationality** and **collective optimality**. In a single-round interaction, the mathematical payoff for "defection" strictly dominates the payoff for "cooperation," regardless of the opponent's choice. This creates a systemic failure where the **Nash Equilibrium**—the point of strategic stability—is located at the least efficient outcome for the group.

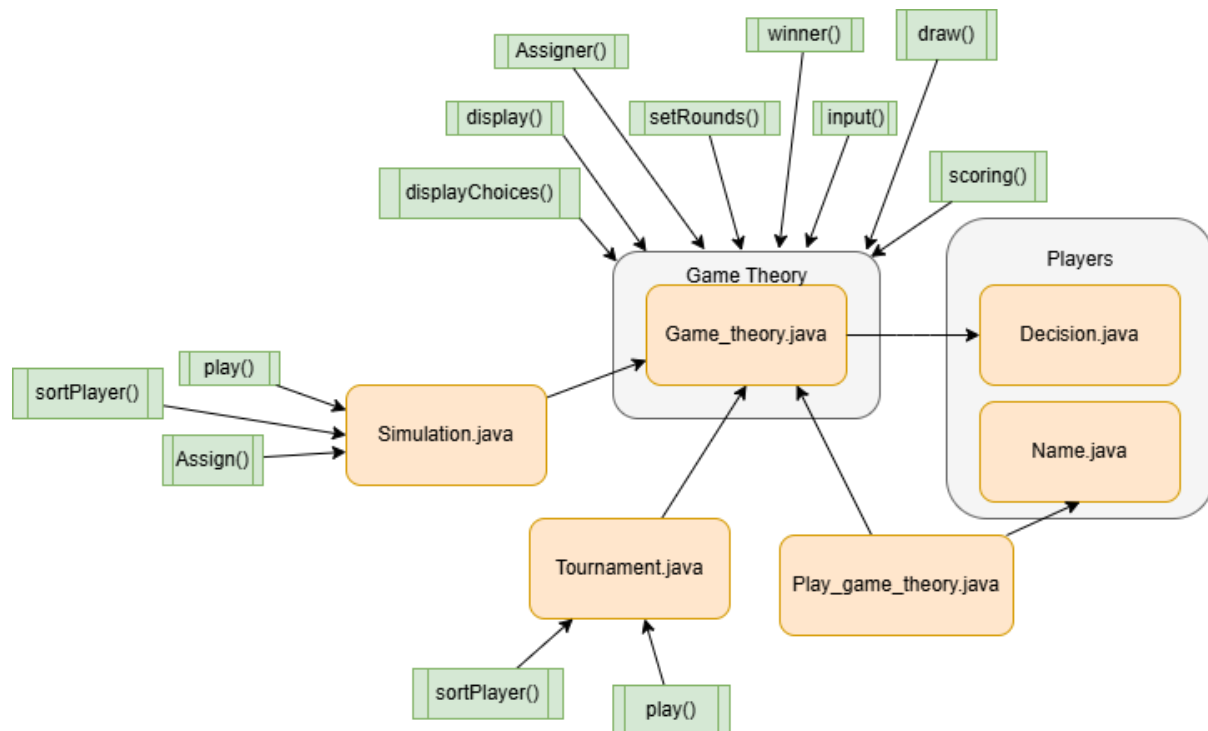
The problem, therefore, is rooted in the mathematical dominance of non-cooperative strategies: a rational actor is incentivized to defect to protect themselves from exploitation or to capture the highest individual reward. When both parties follow this logical path of self-interest, they inevitably fall into a "sub-optimal equilibrium," leaving both sides significantly worse off than if they had coordinated. The core of this dilemma is the inability to escape this trap without verifiable trust or enforceable agreements. We must analyse why the pursuit of the "best" individual choice consistently leads to the "worst" collective failure and identify what mechanisms can shift the incentive structure toward sustainable cooperation.

## Introduction

The **Prisoner's Dilemma** stands as the most influential model in **game theory**, originally formulated by Merrill Flood and Melvin Dresher in 1950 and later formalized by Albert W. Tucker. At its core, the game explores the tension between **individual rationality** and **collective benefit** through a simple matrix: two players must choose between cooperation or defection without knowing the other's intent. While mutual cooperation leads to the best collective outcome, the mathematical incentive for each player is to "defect" to maximize their own gain or minimize their risk. This creates the famous **Nash Equilibrium**, where both parties betray each other and receive a worse result than if they had collaborated. By simplifying complex human behaviour into a set of numeric payoffs, this model has become a vital tool for analysing competition in economics, the ethics of social trust, and even the strategic logic of international arms races.

The study of this dilemma took a revolutionary turn in the early 1980s when political scientist **Robert Axelrod** hosted a series of computer tournaments to find the most effective strategy for the game. Axelrod invited game theorists, economists, and mathematicians from around the world to submit computer programs that would play against one another over hundreds of repeated rounds, known as the **Iterated Prisoner's Dilemma**. To the surprise of many, the winner was the simplest entry: **Tit-for-Tat**, submitted by Anatol Rapoport. This strategy cooperated on the first move and thereafter simply copied its opponent's previous action. Axelrod's tournament proved that "nice" strategies—those that are not the first to defect, are provokable by betrayal, but are also forgiving—can outperform selfish ones in the long run. This discovery shifted the paradigm of game theory, demonstrating that cooperation can evolve and thrive even in a world of self-interested actors.

## Overview



**Fig 1. Block Diagram**

## Tools Used

### Hardware Requirements

- A personal computer or laptop with the following minimum specifications or higher:
- Intel Core i3 processor or higher o 8 GB RAM (minimum 4 GB supported for basic execution)
- 500 GB Hard Disk / SSD storage
- Standard keyboard and mouse
- A stable internet connection (2 Mbps or higher) for development tools, libraries, and documentation access

### Software Requirements

- Operating System
  - Windows 10 / 11, Linux (Ubuntu or similar distributions)
- Development Environment / IDE
  - Visual Studio Code
  - Notepad
- Programming Language
  - Java (JDK 17 or higher)
- Version Control
  - Git and Github

## OOP Concepts Used

### 1. Classes and Objects:

**class** serves as a blueprint or template that defines the structure and behaviour (data fields and methods) common to all entities of a certain type, while an **object** is a specific, concrete instance of that class created during runtime. Think of a class as a conceptual architectural drawing for a house, specifying features like rooms and windows, whereas an object is the actual physical house built from those plans, possessing its own unique address and paint colour. Essentially, the class provides the logic and definition, while the object holds the actual data and executes the tasks defined by the class.

### 2. Encapsulation:

It refers to the bundling of data (attributes) and the methods (functions) that operate on that data into a **single unit**, known as a class. Beyond just organization, encapsulation acts as a protective shield that prevents external code from accessing or modifying the internal state of an object directly.

### 3. Static Members:

Static members are **class-level** variables and methods that are shared by all instances rather than belonging to a specific object. Unlike instance members, which are unique to every object created, only one copy of a static member exists in memory, making it ideal for tracking shared data or providing utility functions. Consequently, these members can be accessed directly using the class name without needing to instantiate an object first.

### 4. Packages:

**Packages** are namespaces used to organize related classes and interfaces into a single group, acting much like folders on a computer to manage large codebases. They help prevent **naming conflicts** by allowing two classes to have the same name as long as they reside in different packages, and they provide an additional layer of **access control** to restrict visibility. By logically grouping code—such as placing all database-related classes in a database package—developers improve maintainability, searchability, and modularity across complex projects.

## Implementation

Strategies Table

| Sno | Strategy Name              | Description                                                                                                                                         |
|-----|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | <b>One man (Cooperate)</b> | Always cooperates. It is a purely "nice" strategy that never initiates a defection.                                                                 |
| 2   | <b>Zero man (Defect)</b>   | Always defects. It aims to maximize its own score by exploiting cooperative players.                                                                |
| 3   | <b>One for Even Man</b>    | A non-adaptive strategy that defects on odd-numbered rounds and cooperates on even-numbered rounds.                                                 |
| 4   | <b>Zero for Even Man</b>   | The inverse of "One for Even Man". It cooperates on odd-numbered rounds and defects on even-numbered rounds.                                        |
| 5   | <b>Player 1100</b>         | Follows a repeating four-round cycle: Cooperate, Cooperate, Defect, Defect.                                                                         |
| 6   | <b>Player 0011</b>         | Follows a repeating four-round cycle: Defect, Defect, Cooperate, Cooperate.                                                                         |
| 7   | <b>Alpha</b>               | Follows a repeating four-round cycle: Defect, Cooperate, Cooperate, Cooperate.                                                                      |
| 8   | <b>Delta</b>               | Follows a repeating four-round cycle: Cooperate, Defect, Defect, Defect.                                                                            |
| 9   | <b>Johan</b>               | A counter to Tit-for-Tat. It does the opposite of the opponent's previous move.                                                                     |
| 10  | <b>Rayon</b>               | A complex, multi-phase strategy. It cooperates for 10 rounds, defects on round 11, then enters a reactive phase based on recent history and scores. |
| 11  | <b>Tit-for-Tat</b>         | Starts by cooperating, then copies the opponent's previous move for all subsequent rounds.                                                          |
| 12  | <b>Tit-for-Two-Tats</b>    | A more forgiving Tit-for-Tat. It only defects if the opponent has defected in the last two consecutive rounds.                                      |
| 13  | <b>Tit-for-Three-Tats</b>  | An even more lenient version. It defects only after the opponent has defected for three consecutive rounds.                                         |

| Sno | Strategy Name                      | Description                                                                                                                                                        |
|-----|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 14  | <b>Two Tit for Tat</b>             | Punishes a defection more severely. When an opponent defects, it retaliates by defecting for the next two rounds.                                                  |
| 15  | <b>Punisher (Grim Trigger)</b>     | Cooperates until the opponent defects once, then defects for the rest of the match.                                                                                |
| 16  | <b>Columbus</b>                    | A variation of Grim Trigger. It switches to permanent defection if the opponent defects for two consecutive rounds.                                                |
| 17  | <b>Galileo</b>                     | Similar to Columbus but more tolerant. It switches to permanent defection if the opponent defects for three consecutive rounds.                                    |
| 18  | <b>Gradual</b>                     | Punishments escalate over time. When the opponent defects for the Nth time, it retaliates by defecting for N rounds, then returns to cooperating.                  |
| 19  | <b>Sharp</b>                       | Makes a decision based on the first round. If the opponent cooperates on round 1, it always cooperates. If the opponent defects on round 1, it always defects.     |
| 20  | <b>Sure</b>                        | Cooperates for the first two rounds. Afterwards, if the opponent has defected in the last two rounds AND has a higher score, it defects. Otherwise, it cooperates. |
| 21  | <b>Sigma</b>                       | Cooperates for the first three rounds. After that, it defects if the opponent's score is higher than its own, otherwise it cooperates.                             |
| 22  | <b>Joss (Win-Stay, Lose-Shift)</b> | Also known as Pavlov. It repeats its last move if it resulted in a high score (3 or 5 points), and switches its move if it resulted in a low score (0 or 1 point). |
| 23  | <b>Random</b>                      | Makes a random choice each round with a 50% chance of cooperating and a 50% chance of defecting.                                                                   |

## Code

In **Decision.java**, different strategies are implemented using public methods

```
1 public static int Tic_for_Tac(int[][] choices,int player_no,int round){//good player
2     if(round==0){
3         return 1;
4     }
5     int player=player_no==1?0:1;
6     return choices[player][round-1];
7 }
8 public static int Tic_for_2_Tac(int[][] choices,int player_no,int round){//good player
9     if(round<2){
10        return 1;
11    }
12    int player=player_no==1?0:1;
13    if(choices[player][round-1]+choices[player][round-2]==0){
14        return 0;
15    }
16    return 1;
17 }
18 public static int Tic_for_3_Tac(int[][] choices,int player_no,int round){//good player
19     if(round<3){
20        return 1;
21    }
22    int player=player_no==1?0:1;
23    if(choices[player][round-1]+choices[player][round-2]+choices[player][round-3]==0){
24        return 0;
25    }
26    return 1;
27 }
```

Fig 2. Sample strategy methods

In **Game\_theory.java**, the Assigner method assigns decisions using player\_no and scoring() method assigns scored for each round.

```
1 public static int Assigner(int player,int player_no,int round){
2     int choice=switch (player) {
3         case 1 -> Decision.One_man();
4         case 2 -> Decision.Zero_man();
5         case 3 -> Decision.One_for_even_man(round);
6         case 4 -> Decision.Zero_for_even_man(round);
7         case 5 -> Decision.Tic_for_Tac(Game_theory.choices,player_no,round);
8         case 6 -> Decision.Punisher(Game_theory.choices,player_no,round);
9         case 7 -> Decision.Sure(Game_theory.choices, player_no, round, Game_theory.score);
10        case 8 -> Decision.Player_1100(round);
11        case 9 -> Decision.Player_0011(round);
12        case 10 -> Decision.Alpha(round);
13        case 11 -> Decision.Delta(round);
14        case 12 -> Decision.Sigma(Game_theory.choices,player_no,round,Game_theory.score);
15        case 13 -> Decision.Tic_for_2_Tac(Game_theory.choices, player_no, round);
16        case 14 -> Decision.Columbus(Game_theory.choices, player_no, round);
17        case 15 -> Decision.Loss(Game_theory.choices, player_no, round);
18        case 16 -> Decision.Johan(Game_theory.choices, player_no, round);
19        case 17 -> Decision.Tic_for_3_Tac(Game_theory.choices, player_no, round);
20        case 18 -> Decision.Galileo(Game_theory.choices, player_no, round);
21        case 19 -> Decision.Sharp(Game_theory.choices, player_no, round);
22        case 20 -> Decision.Rayon(Game_theory.choices, player_no, round, Game_theory.score);
23        case 21 -> Decision.Gradual(Game_theory.choices, player_no, round);
24        case 22 -> Decision.Two_tic_for_tac(Game_theory.choices, player_no, round);
25        case 23 -> Decision.Random_man();
26        default ->1;
27    };
28    return choice;
29 }
```

Fig 3. Assinger() method

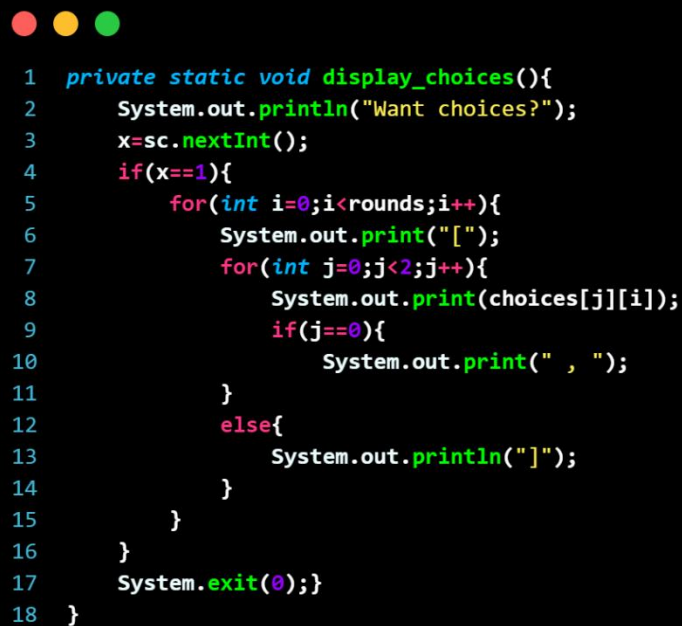
The scoring() scores the two players immediately in a round.



```
1 public static void scoring(int round){
2     if(choices[0][round]==choices[1][round]){
3         if(choices[0][round]==1){
4             score[0]+=3;
5             score[1]+=3;
6         }
7         else{
8             score[0]+=1;
9             score[1]+=1;
10        }
11    }
12    else if(choices[0][round]<choices[1][round]){
13        score[0]+=5;
14    }
15    else if(choices[1][round]<choices[0][round]){
16        score[1]+=5;
17    }
18 }
```

Fig 4. scoring() method

The display\_choices() method displays the choices made by players throughout the game



```
1 private static void display_choices(){
2     System.out.println("Want choices?");
3     x=sc.nextInt();
4     if(x==1){
5         for(int i=0;i<rounds;i++){
6             System.out.print("[");
7             for(int j=0;j<2;j++){
8                 System.out.print(choices[j][i]);
9                 if(j==0){
10                    System.out.print(" , ");
11                }
12                else{
13                    System.out.println("]");
14                }
15            }
16        }
17        System.exit(0);}
18 }
```

Fig 5. display\_choices() method

In `Play_game_theory.java` is a playground to test new players and analyse their decisions.

```
1  String[] players={"One_man","Zero_man","One_for_even_man","Zero_for_even_man",
2  "Tic_for_tac","Punishner","Sure","Player_1100","Player_0011","Alpha","Delta",
3  "Sigma","Tic_for_2_tac","Colombus","Joss","Johan","Tic_for_3_tac","Galileo",
4  "Sharp","Rayon","Gradual","Two_tic_for_tac","Random_man"};
5  int[] player=new int[2];
6  for(int k=0;k<2;k++){
7      if(k==0){
8          System.out.println("Choose player "+(k+1)+" (0 if you want to play)");
9          for(int i=0;i<players.length;i++){
10             System.out.println(players[i]+"....."+(i+1));
11         }
12     }
13     //player[k]=Game_theory.random_int_chooser(players.Length);
14     //player[k]=k==0?15:sc.nextInt();
15     //player[k]=k==0?0:Game_theory.random_int_chooser(16);
16     //System.out.println(player[k]);
17     player[k]=sc.nextInt();
18 }
19 int[] choice=new int[2];
20
21 for(int i=0;i<Game_theory.rounnds;i++){
22     for(int k=0;k<2;k++){
23         int User_input;
24         if(player[k]==0){
25             //System.out.println("Function check 1");
26             System.out.println("Round "+(i+1));
27             User_input=sc.nextInt();
28             choice[k]=User_input;
29         }else{
30             choice[k]=Game_theory.Assigner(player[k],k,i);
31         }
32     }
33     Game_theory.input(i,choice[0],choice[1]);
34     Game_theory.scoring(i);
35 }
```

Fig 6. `Play_game_theory.java`

In **Tournament.java**, play() method is used to make two players play against each other.



```
1  public static void play(int player_1,int player_2){
2      for(int round=0;round<Game_theory. rounds;round++){
3          int choice_1=Game_theory.Assigner(player_1, 0, round);
4          int choice_2=Game_theory.Assigner(player_2, 1, round);
5          Game_theory.input(round, choice_1, choice_2);
6          Game_theory.scoring(round);
7      }
8  }
9  public static void main(String[] args) {
10     for(int i=0;i<players_count;i++){
11         for(int j=i;j<players_count;j++){
12             Game_theory.score[0]=0;
13             Game_theory.score[1]=0;
14             play(i+1,j+1);
15             player_score[i]+=Game_theory.score[0];
16             player_score[j]+=Game_theory.score[1];
17         }
18     }
```

Fig 7. Tournament.java

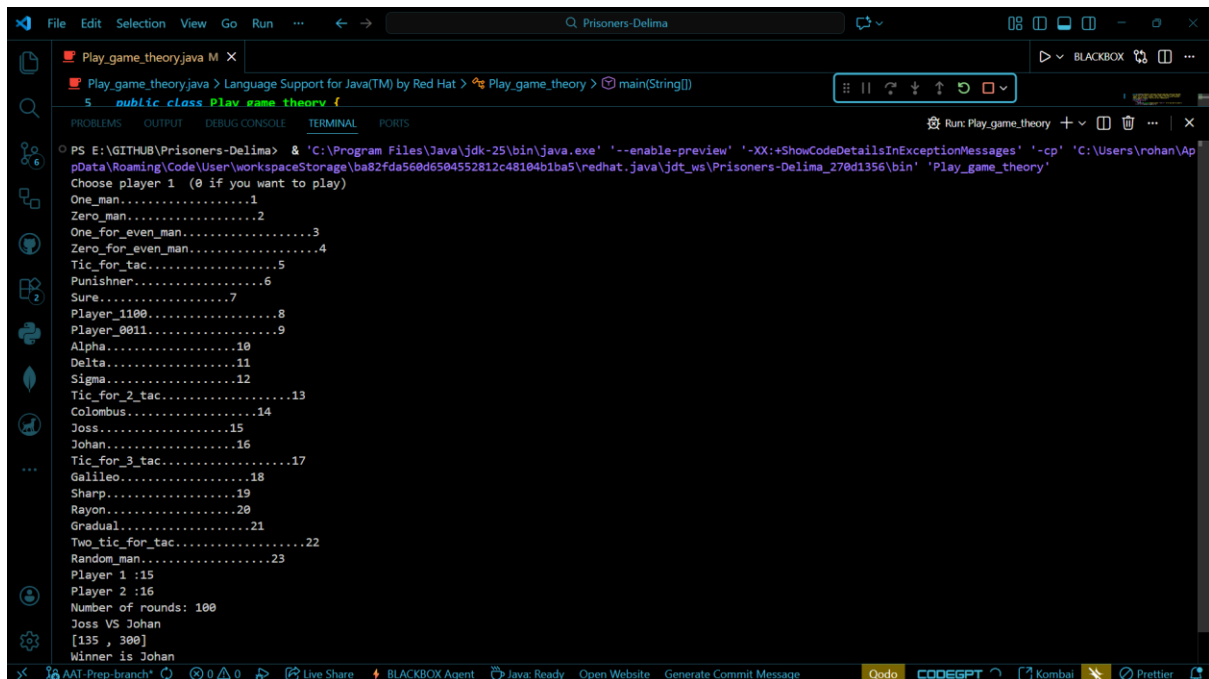
In **Simulation.java**, play() and Assign() methods are used to set children for players and assign wins and loses.

```
1  public static void play(int player_1,int player_2){
2      for(int round=0;round<Game_theory.rounnds;round++){
3          int choice_1=Game_theory.Assigner(player_1, 0, round);
4          player_children[player_1-1]+=(choice_1==1)?1:-1;
5          int choice_2=Game_theory.Assigner(player_2, 1, round);
6          player_children[player_2-1]+=(choice_2==1)?1:-1;
7          Game_theory.input(round, choice_1, choice_2);
8          Game_theory.scoring(round);
9      }
10 }
11 public static void Assign(int i,int a,int j,int b){
12     if(a>b){
13         player_wins[i]+=1;
14         player_loses[j]+=1;
15     }else{
16         if(a<b){
17             player_wins[j]+=1;
18             player_loses[i]+=1;
19         }
20         else if(a==b){
21             player_wins[i]+=1;
22             player_wins[j]+=1;
23         }
24     }
25 }
```

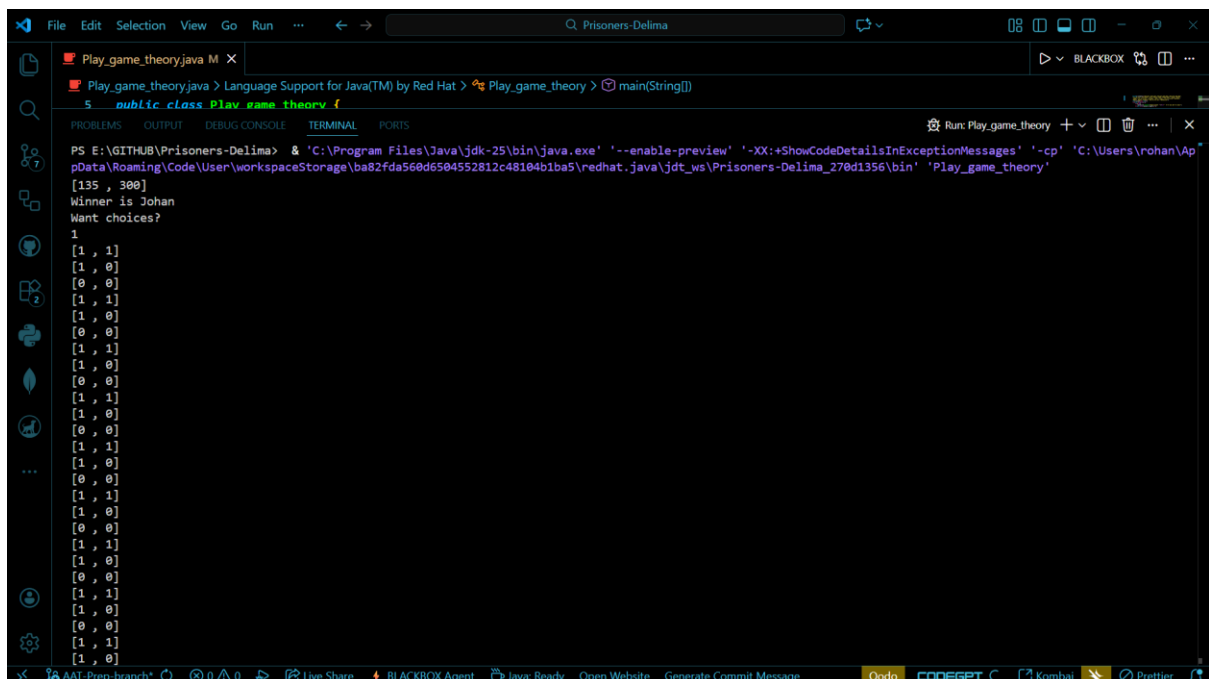
Fig 8. Simulation.java

## Output

Play\_game\_theory.java

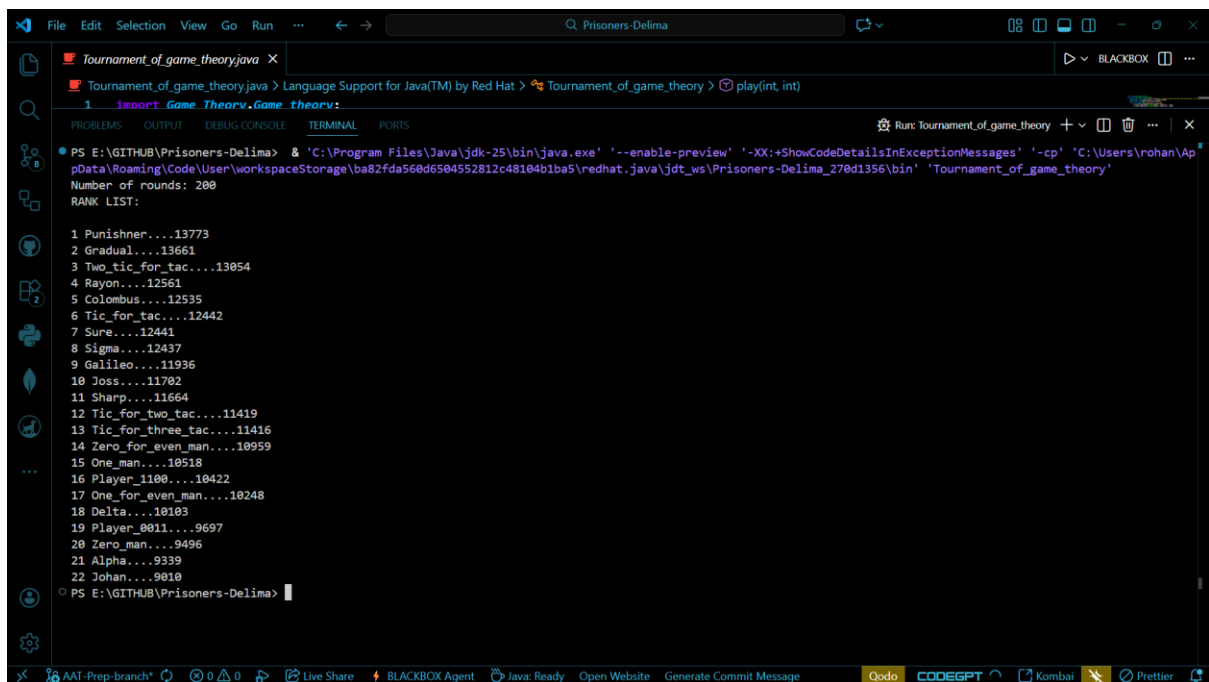


**Fig 9. User can make any two players play for desired number of rounds**



**Fig 10. User can view the choices the players took for each round for further study**

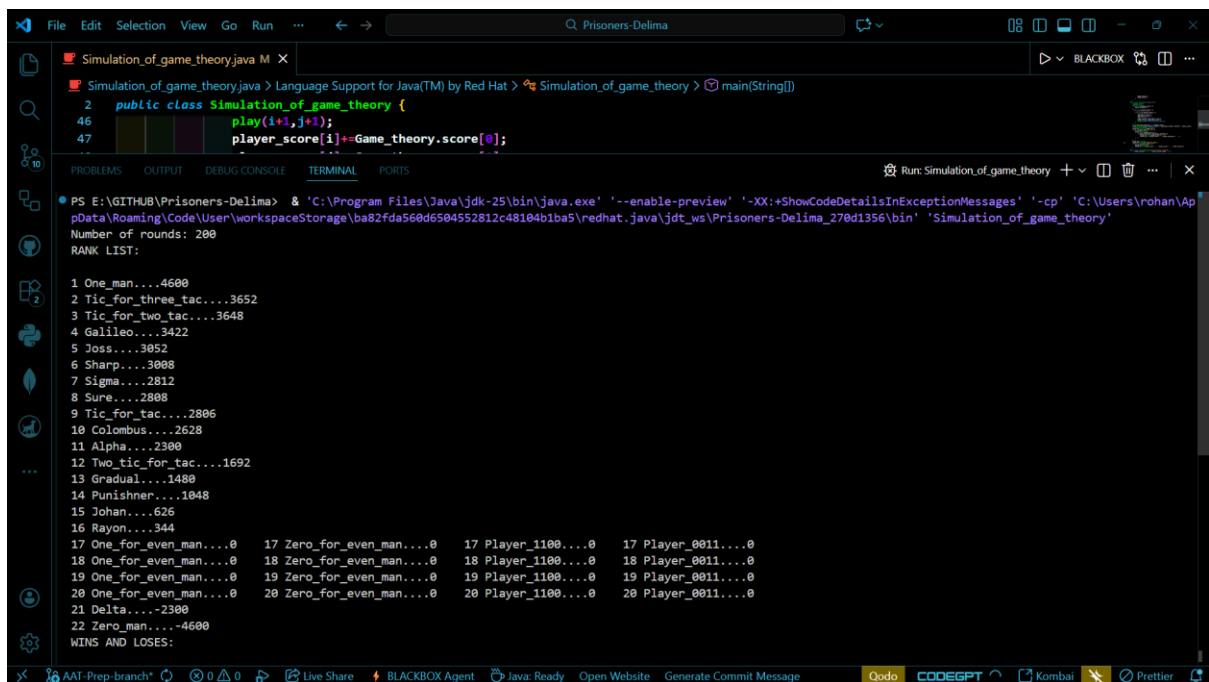
## Tournament.java



```
PS E:\GITHUB\Prisoners-Delima> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\rohan\AppData\Roaming\Code\User\workspaceStorage\ba82fda560d6504552812c48104b1ba5\redhat.java\jdt_ws\Prisoners-Delima_270d1356\bin' 'Tournament_of_game_theory'
Number of rounds: 200
RANK LIST:
1 Punisher....13773
2 Gradual....13661
3 Two_tic_for_tac....13054
4 Rayon....12561
5 Columbus....12535
6 Tic_for_tac....12442
7 Sure....12441
8 Sigma....12437
9 Galileo....11936
10 Joss....11702
11 Sharp....11664
12 Tic_for_two_tac....11419
13 Tic_for_three_tac....11416
14 Zero_for_even_man....10959
15 One_man....10518
16 Player_1100....10422
17 One_for_even_man....10248
18 Delta....10103
19 Player_0011....9697
20 Zero_man....9496
21 Alpha....9339
22 Johan....9010
PS E:\GITHUB\Prisoners-Delima>
```

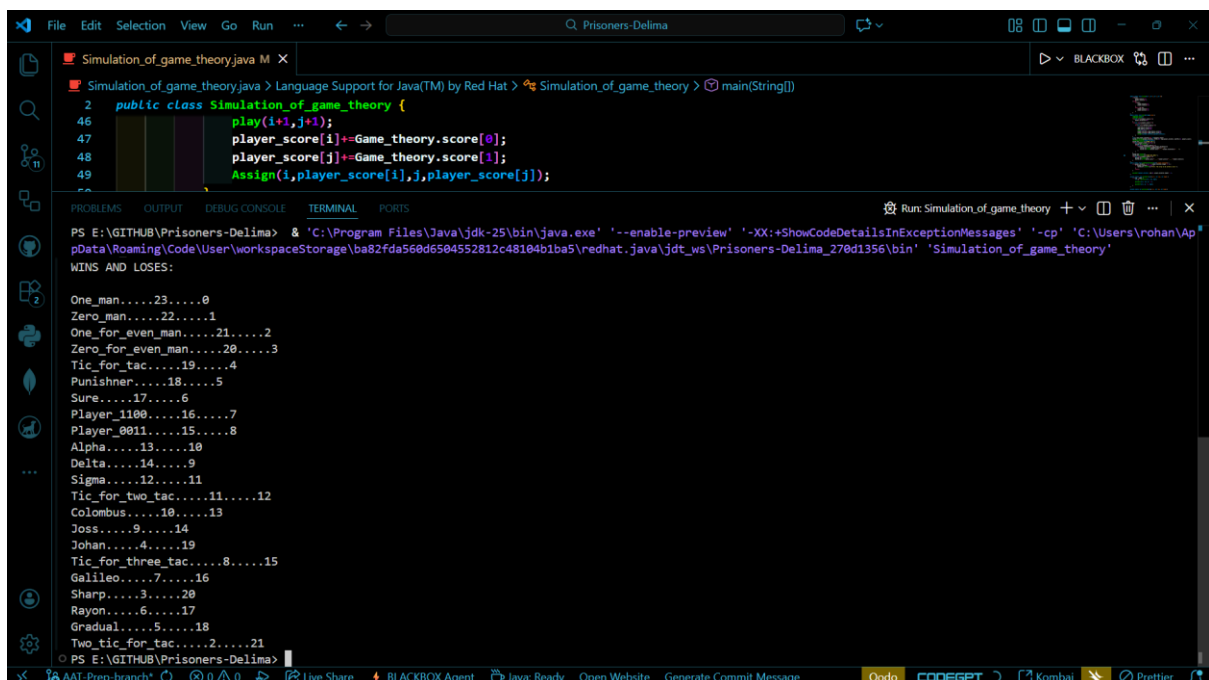
**Fig 11. User can make all the players play with each other for a desired number of rounds**

## Simulation.java



```
PS E:\GITHUB\Prisoners-Delima> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\rohan\AppData\Roaming\Code\User\workspaceStorage\ba82fda560d6504552812c48104b1ba5\redhat.java\jdt_ws\Prisoners-Delima_270d1356\bin' 'Simulation_of_game_theory'
Number of rounds: 200
RANK LIST:
1 One_man....4600
2 Tic_for_three_tac....3652
3 Tic_for_two_tac....3648
4 Galileo....3422
5 Joss....3052
6 Sharp....3008
7 Sigma....2812
8 Sure....2808
9 Tic_for_tac....2806
10 Columbus....2628
11 Alpha....2300
12 Two_tic_for_tac....1692
13 Gradual....1480
14 Punishner....1048
15 Johan....626
16 Rayon....344
17 One_for_even_man....0
18 Zero_for_even_man....0
19 One_for_even_man....0
20 One_for_even_man....0
21 Delta....-2300
22 Zero_man....-4600
WINS AND LOSSES:
17 Zero_for_even_man....0
18 Player_1100....0
19 Player_1100....0
20 Player_0011....0
18 Player_1100....0
19 Player_1100....0
19 Player_0011....0
20 Player_1100....0
20 Player_0011....0
```

Fig 12. Rank list of players based on their performance is listed



```
PS E:\GITHUB\Prisoners-Delima> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\rohan\AppData\Roaming\Code\User\workspaceStorage\ba82fda560d6504552812c48104b1ba5\redhat.java\jdt_ws\Prisoners-Delima_270d1356\bin' 'Simulation_of_game_theory'
WINS AND LOSSES:
One_man....23.....0
Zero_man....22.....1
One_for_even_man....21.....2
Zero_for_even_man....20.....3
Tic_for_tac....19.....4
Punishner....18.....5
Sure....17.....6
Player_1100....16.....7
Player_0011....15.....8
Alpha....13.....10
Delta....14.....9
Sigma....12.....11
Tic_for_two_tac....11.....12
Columbus....10.....13
Joss....9.....14
Johan....4.....19
Tic_for_three_tac....8.....15
Galileo....7.....16
Sharp....3.....20
Rayon....6.....17
Gradual....5.....18
Two_tic_for_tac....2.....21
```

Fig 13. Number of rounds won and lost is also displayed

## References

### Github repository:

- <https://github.com/Rohan-Shridhar/Prisoners-Delima>

### Front end site:

- <https://rohan-shridhar.github.io/Prisoners-Delima/>

### Wikipedia blog:

- [https://en.wikipedia.org/wiki/Game\\_theory](https://en.wikipedia.org/wiki/Game_theory)
- [https://en.wikipedia.org/wiki/Prisoner%27s dilemma](https://en.wikipedia.org/wiki/Prisoner%27s_dilemma)

### Axelrod Tournament:

- <https://cs.stanford.edu/people/eroberts/courses/soco/projects/1998-99/game-theory/axelrod.html>